Recitation 3 – HTTP and Web Performance

Daniel Karalnik and Liam Tal

Computer Networks

Table of Contents

1. HTML and Browsers
2. Round Trip Time
3. HTTP Connection Types
4. RTT Calculation Exercises
5. Additional Practice Problems

HTML

- Markup Language
  - Using annotation
  - What other markup languages do you know?

- Example:

```
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>

<h1>This is a Heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

Browser

- Interpret HTML & display
- Using a Layout Engine
  - WebKit (Safari)
  - Blink (Chrome/Opera/new Edge)
  - Trident $\rightarrow$ EdgeHTML (IE/Edge)
  - Gecko (Firefox)
- Different feature support & implementation
  - Is that a problem? why?

Challenge: Different engines may render the same HTML differently! Web developers must test across multiple browsers.

HTML: Homework Tips!

- HTML is not case sensitive (`<tag>` == `<TAG>` == `<TaG>`)
- You can use any text editor. I recommend:
  - Visual Studio Code (https://code.visualstudio.com)
- Great HTML tutorial: http://www.w3schools.com/html
- List of HTML tags: http://www.w3schools.com/tags/default.asp
- HTML Forms: http://www.w3schools.com/html/html_forms.asp

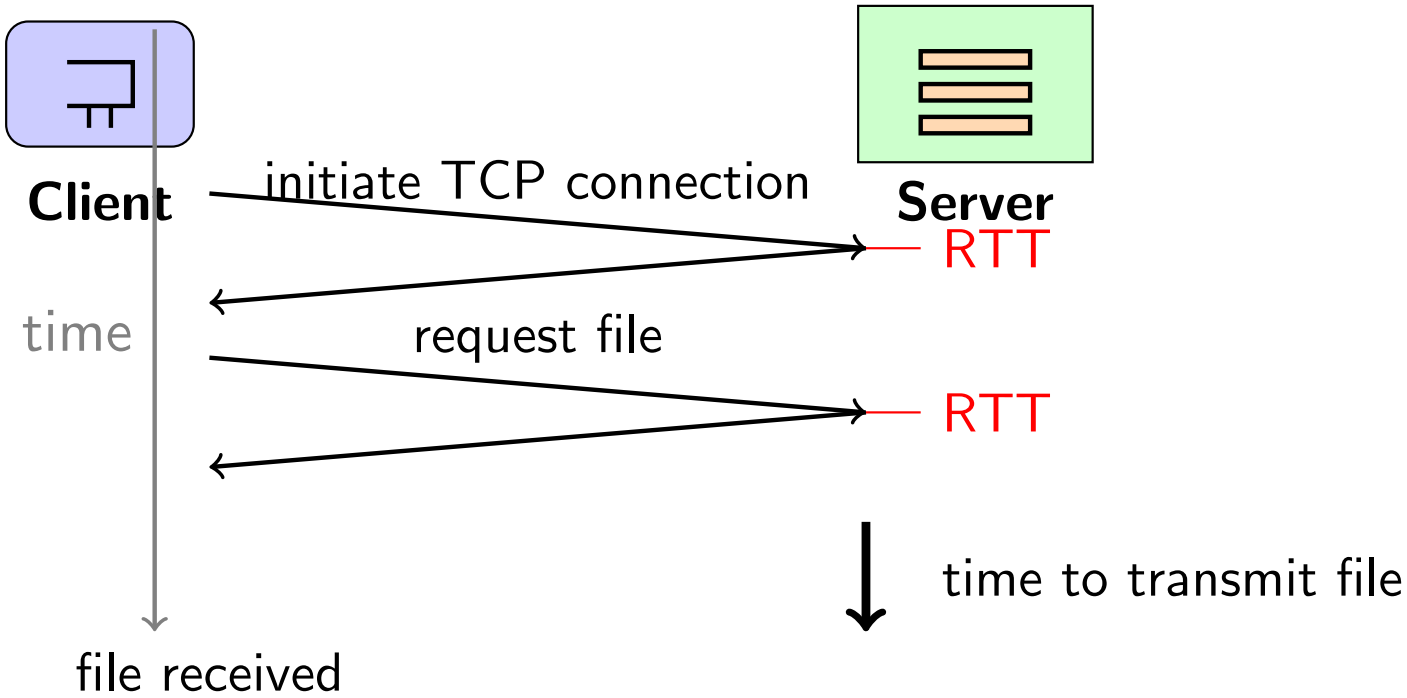
Tool Demo: Chrome Developer Tools

Context: Demonstration of basic network inspection concepts when loading a site (e.g., https://www.ynet.co.il).

1. **Capture traffic for a site**
   - Open Chrome → DevTools (F12 / Ctrl+Shift+I).
   - Switch to the **Network** tab and reload the page.
   - Observe total request count, size, and load time in the Network footer.
2. **Inspect decrypted request & response details**
   - Click any request to open **Headers**, **Preview**, **Response**, and **Timing**.
   - Chrome DevTools displays request and response data after TLS termination.

Round Trip Time (RTT): Definition

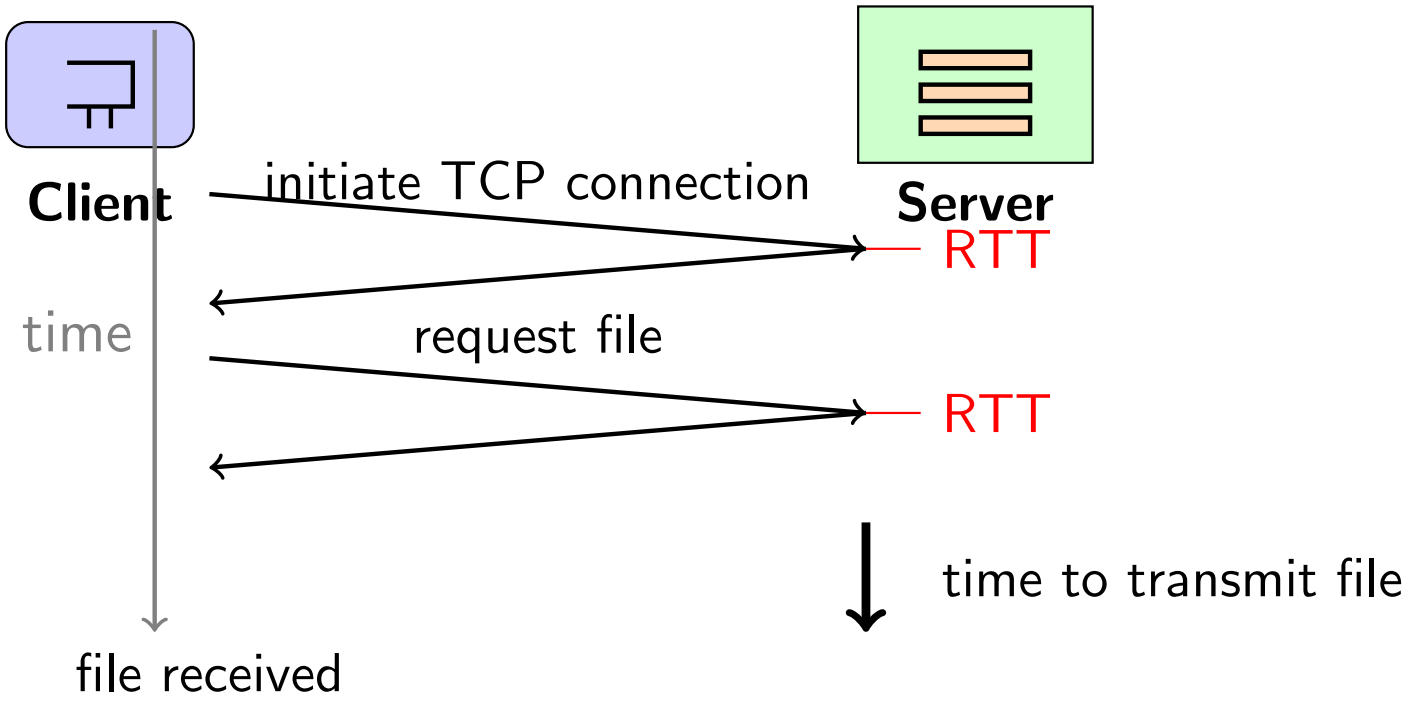
The time it takes for a small packet to travel from client to server and then back to the client.



What is the response time in the figure above in terms of RTT, R and L (file size)?

Round Trip Time (RTT): Calculation

The time it takes for a small packet to travel from client to server and then back to the client.



Round Trip Time (RTT): Calculation

The time it takes for a small packet to travel from client to server and then back to the client.

Answer:

$$\boxed{2 \times \text{RTT} + L/R}$$

Why is RTT Important?

- When you upgrade your home internet connection from 5Mbps to 10Mbps, will web pages load twice as fast? Why?
- When is it a good idea to increase your bandwidth?

Key Insight

Modern web pages contain **many objects** (100+ on average). Each object may require multiple round trips. This means **latency (RTT) often matters more than bandwidth!**

Bandwidth vs. Latency: Mike Belshe Research

Bandwidth Impact

Bandwidth	Load Time (ms)
1Mbps	3106
2Mbps	1950
5Mbps	1443
10Mbps	1360

Improvement: 1Mbps $\rightarrow$ 10Mbps
56% reduction

RTT Impact

RTT	Load Time (ms)
240ms	3964
120ms	2291
60ms	1450
20ms	981

Improvement: 240ms $\rightarrow$ 20ms
75% reduction

Conclusion: Reducing RTT has **much greater impact** than increasing bandwidth!

Optimizing Page Load Time

So, how can we speed up single page load time?

Strategy 1: Decrease RTT

- On the last mile
- Bring content closer to user (CDN)
- Optimize routing

Strategy 2: Reduce Round Trips

- Parallel connections
- Persistent connections
- HTTP Pipelining
- Minimize redirects & DNS lookups
- Use QUIC instead of TCP

Focus of this recitation: Strategy 2 techniques!

HTTP Connection Types: Recap

- Parallel connections
  - ?
- Persistent connections
  - ?
- HTTP Pipelining
  - ?

HTTP Connection Types: Parallel Connections

- Parallel connections
  - Multiple TCP connections (most browsers do that)
- Persistent connections
  - ?
- HTTP Pipelining
  - ?

HTTP Connection Types: Persistent Connections

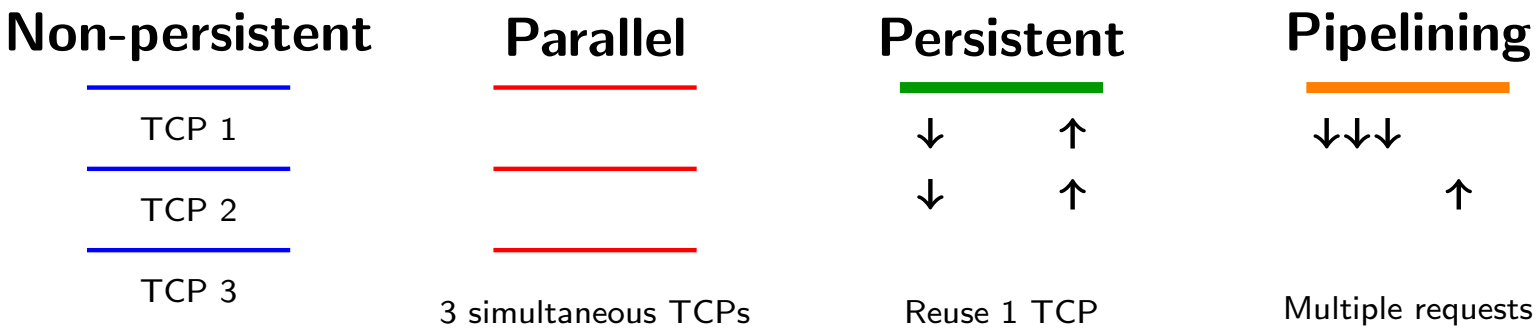
- Parallel connections
  - Multiple TCP connections (most browsers do that)
- Persistent connections
  - Re-using the same TCP connection for multiple requests
- HTTP Pipelining
  - ?

HTTP Connection Types: Pipelining

- Parallel connections
  - Multiple TCP connections (most browsers do that)
- Persistent connections
  - Re-using the same TCP connection for multiple requests
- HTTP Pipelining
  - Sending multiple requests without waiting for a response
  - Is it possible to have a Non-persistent connection with pipelining?

Answer: No! Pipelining requires a persistent connection because you need to send multiple requests over the *same* connection without closing it.

Visual Comparison: Connection Types



Pipelining = Persistent + sending requests without waiting for responses

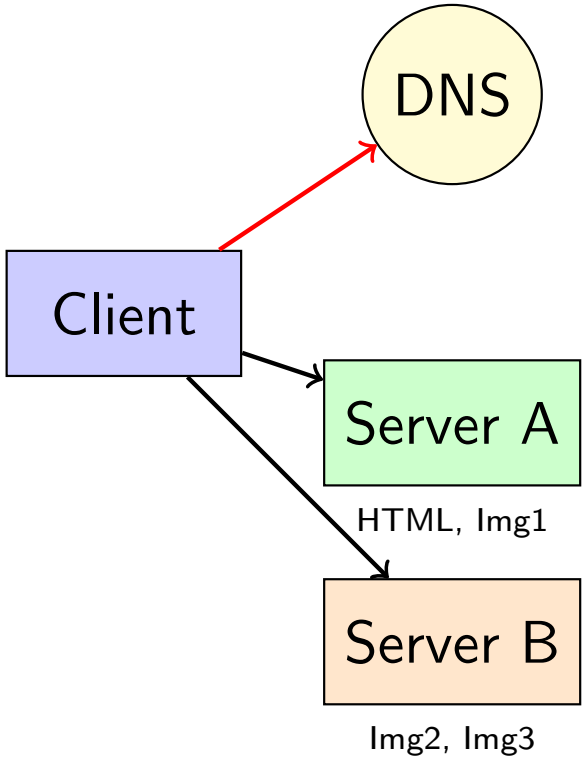
Exercise Setup

Scenario:

- Server A: HTML + Image 1
- Server B: Images 2 & 3
- IP of A: unknown (need DNS)
- IP of B: known

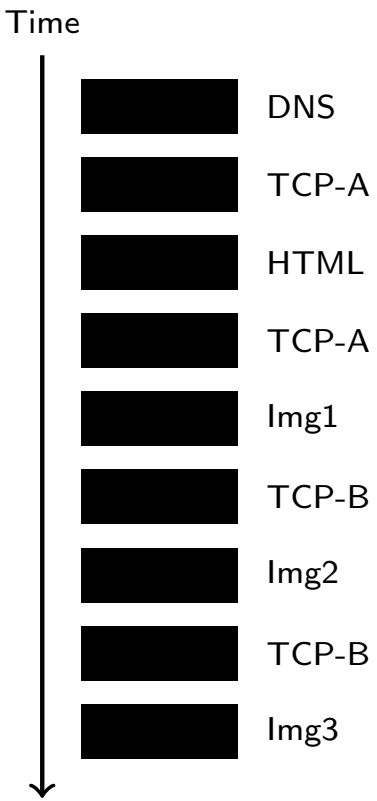
Question: How many RTTs?

Non-persistent,
No parallel



Solution: Non-persistent, No Parallel

1. DNS query for A's IP
2. TCP with A
3. HTTP req/resp HTML
4. TCP with A
5. HTTP req/resp Image 1
6. TCP with B
7. HTTP req/resp Image 2
8. TCP with B
9. HTTP req/resp Image 3



Result

9 RTT

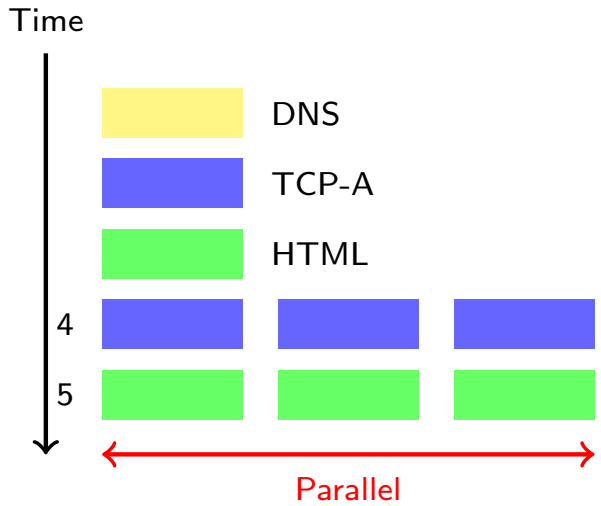
Exercise Variations: Connection Types

Variation	RTT	Key Steps
Non-persistent + Parallel	**5**	All 3 images in parallel (step 4)
Persistent + No pipeline	**7**	Sequential on both connections
Persistent + Pipeline	**6**	Pipeline only on Server B
Persistent + Pipeline + Parallel	**5**	Both parallel & pipeline

Let's work through each variation in detail...

Solution: Non-persistent with Parallel

1. DNS query for A's IP
2. TCP with A
3. HTTP req/resp HTML
4. In Parallel
   - TCP with A
   - TCP with B
   - TCP with B
5. In Parallel
   - HTTP Img 1
   - HTTP Img 2
   - HTTP Img 3



Result

5 RTT

Solution: Persistent without Pipelining

1. DNS query for A's IP
2. TCP with A
3. HTTP req/resp HTML
4. HTTP Img 1 (using 2)
5. TCP with B
6. HTTP Img 2 (using 5)
7. HTTP Img 3 (using 5)

Result

7 RTT

Formula Ref

Persistent: Reuse TCP
Steps 6-7 use connection 5

Solution: Persistent with Pipelining

1. DNS query for A's IP
2. TCP with A
3. HTTP req/resp HTML
4. HTTP Img 1 (using 2)
5. TCP with B
6. In pipeline
   - HTTP Img 2 (using 5)
   - HTTP Img 3 (using 5)

Result

6 RTT

Formula Ref

Pipeline: Send without waiting
Imgs 2-3 pipelined on B

Solution: Persistent with Pipelining and Parallel

1. DNS query for A's IP
2. TCP with A
3. HTTP req/resp HTML
4. In Parallel
   - HTTP Img 1 (using 2)
   - TCP with B
5. In pipeline
   - HTTP Img 2 (using 5)
   - HTTP Img 3 (using 5)

Result

5 RTT

Formula Ref

Best: Parallel + Pipeline
Fastest possible!

Summary: RTT Comparison for All Scenarios

Scenario:1 HTML + 3 images (1 on Server A, 2 on Server B), DNS needed for A

Connection Type	RTT Count	Improvement
Non-persistent, No parallel	9	Baseline
Non-persistent, Parallel	5	44% faster
Persistent, No pipeline, No parallel	7	22% faster
Persistent, Pipeline, No parallel	6	33% faster
Persistent, Pipeline, Parallel	**5**	**44% faster**

Key Takeaway: Combining persistent connections, pipelining, and parallel connections gives the best performance!

Problem: Multi-Server HTTP Scenario

Suppose the **HTML page** your browser wants to download is **20K bits** long, and contains **5 embedded images**, each of **50K bits** in length. The page is stored on **server A** and the 5 images are all stored on **server B**. The **RTT** from your browser to **server A is 500 msec**, and to **server B is 200 msec**. The network paths between your browser and servers A and B are two **1000Mbps** links. Assume:

- The time it takes to transmit a GET message is zero
- You should account for the time it takes to transmit the HTML/image file
- Your browser knows the exact IP address of the servers in advance.

Assuming **non-persistent HTTP**, with **no parallel** connections. What is the response time from the point when the user requests the page to the point in time when the page and its embedded objects are displayed?

Solution Approach

Question 1

How long does it take to fetch the HTML?

Question 2

How long does it take to fetch an image?

Hint

Use: **RTT** (round trip time) + **Transmission delay** (L/R)
Remember: Each non-persistent connection needs 2 RTT (TCP setup + HTTP request/response)

Solution: Step-by-Step Calculation

HTML Fetch Time

Server A (RTT = 500ms)

- 2 RTT $= 2 \times 500$ ms
- Transmission $= \frac{20{,}000}{10^9}$ $= 0.02$ ms

Total: 1000.02 ms

Image Fetch Time

Server B (RTT = 200ms)

- 2 RTT $= 2 \times 200$ ms
- Transmission $= \frac{50{,}000}{10^9}$ $= 0.05$ ms

Total: 400.05 ms

Next: Calculate total time for 1 HTML + 5 images

Solution: Final Answer

Total Time = HTML + 5 $\times$ Images

$$= 1000.02 + 5 \times 400.05$$

$$= 1000.02 + 2000.25$$

$$= 3000.27 \text{ ms}$$

Answer: 3000.27 msec

or approximately **3 seconds**